

EDM
HOLDINGS

EDM OS Build Plan

Internal-first, product-optional. The decision, the build sequence, and the disciplines that keep the option open.

EDM OS — construction operating system · UAE fit-out & drywall

Decision: build for EDM first; preserve the option to sell, cheaply

Companion: the **edm-os** repository (now with the configuration layer implemented)

Internal-first, product-optional.

Build EDM OS to run EDM Holdings. Keep — at near-zero extra cost — the option to sell it to other UAE fit-out contractors later. Do not build product machinery on spec, and do not treat "it feels finished" as the trigger to sell.

The trap to avoid. "We'll sell it once it's perfect for us" is backwards. Perfect-for-EDM means shaped entirely around one company's quirks — which is exactly what makes software **unsellable** to anyone else. You don't productise when it's polished; you productise when other contractors prove they have the same pain and will pay.

The principle: buy the cheap 20% now, defer the expensive 80%

Pay for now (cheap, preserves the option)

Multi-tenant data, clean module boundaries, and config-driven behaviour — trades, workflows and features as data, not code. Costs almost nothing if done from the start; impossible to retrofit cleanly later.

Defer until a buyer appears (expensive, product-only)

Billing, self-serve sign-up, super-admin console, marketing site, support tooling, multi-org provisioning. Real money, zero value to EDM, and premature without a paying customer.

The trigger to revisit selling

Not a feeling — a signal. Recommended: **three other UAE contractors tell you, unprompted, that they would pay for this.** Until then, every hour goes into making EDM OS excellent for EDM. And know what selling actually is: founding a second company — support, sales, onboarding, churn, SLAs, securing other firms' data — not flipping a switch.

Build what you win, when you win it.

EDM is pre-revenue with no live projects, so don't build fifteen modules up front. Build the four you'll actually touch as you bid and win your first jobs — and let real use shape the rest. The order follows the money.

ORDER	MODULE	BUILD WHEN	STATUS
0	Auth, RBAC, Settings, Dashboard shell	Foundation — before anything else.	IN REPO
1	Tenders + Estimating	The moment you start bidding — you can't quote without them. First revenue activity.	TENDERS IN REPO
2	Projects + Site reporting	The moment you win the first job — to run programme and capture site daily.	PROJECTS IN REPO
3	CRM	Alongside 1-2 — light at first (companies, contacts, follow-ups); deepen as the pipeline grows.	NEXT
4	Variations + RFIs	As the first projects generate change and queries — usually weeks in.	PHASE 2
5	Quality + HSE + Trade modules	As projects mature and you need inspections, snags, safety records and m ² -installed tracking.	PHASE 2
6	Procurement + Documents + Mobile	When buying and document volume justify it; mobile when site teams need offline capture.	PHASE 2
7	Financials + AI suite	Once there's enough live data to forecast and enough scale to benefit from AI assistants.	PHASE 3

The rule: build what you need **this quarter**, against real projects — not the cathedral against a wishlist. Each module follows the Tenders/Projects pattern already in the repo, so each is days, not months.

Cheap to hold, costly to retrofit.

These cost almost nothing while building, and they are the entire difference between a tool that can become a product and one that can't. Several are already implemented in the foundation.

DISCIPLINE	WHAT IT MEANS IN PRACTICE	STATE
Multi-tenant always	Every table scoped by <code>organisationId</code> ; queries use <code>tenantWhere()</code> . Never a query that could cross orgs.	DONE
Config, not code	Trades, workflow stages, fiscal defaults live in <code>OrganisationSettings</code> — no "how EDM works" hardcoded.	DONE
Feature flags	<code>@Feature("...")</code> gates modules per org. Phase-1 on, later modules dark. Doubles as SaaS tiering.	DONE
Clean module boundaries	Each module self-contained (controller/service/DTOs); no tangled cross-imports.	DONE
Shared types	DTOs/types shared so web, API and (later) mobile never drift.	ONGOING
Per-org branding slot	<code>OrganisationSettings.branding</code> reserved for white-labelling later.	RESERVED
Audit everything	<code>AuditLog</code> on create/update/delete/export — needed internally, essential for selling.	MODELLED

What changed in the repo today: the configuration layer is now real — `OrganisationSettings` in the schema, `org-config.ts` with EDM's defaults, a `/settings` API, and the `@Feature` flag guard wired globally and demonstrated on Tenders. The option to sell is now preserved in code, not just intention.

Don't furnish a house nobody's bought.

This is product-only machinery. It is expensive, it adds nothing for EDM, and building it before a paying customer is how SaaS money gets burned. Add each item the day it's genuinely needed.

Billing & subscriptions — payment, plans, invoicing, dunning. Build when you have someone to bill.

Self-serve sign-up & onboarding — public registration, guided setup, trials. Build when you're acquiring customers, not before.

Super-admin console — cross-org management, impersonation, support views. Build when you're supporting more than one org.

Marketing site & sales funnel — a product needs its own go-to-market; that's a separate effort entirely.

Customer support tooling — ticketing, help docs, status page, SLAs.

Multi-org provisioning & data isolation hardening — the schema is ready; the operational machinery isn't needed until org two.

The discipline: the data model already supports all of this (it's multi-tenant by design). So deferring costs you nothing — you are not painting yourself into a corner, you are simply not building furniture for an empty house.

The near-term, concretely.

WHEN	DO
Now	Stand the foundation up: provision Supabase, run <code>migrate + seed</code> , deploy web to Vercel and the API to a small host. You have a live, multi-tenant shell with auth, settings and a dashboard.
As you bid	Finish Estimating (the repo has Tenders; Estimating is next) so you can price real tenders in the system. This is the first place EDM OS earns its keep.
On first win	Use Projects + Site reporting on a real job. Capture daily from day one — the same habit that feeds your photography and content engines.
Each quarter	Add the next module the live work demands, following the reference pattern. Review the build order against what projects actually need.
Watch for	The selling signal — three contractors who'd pay. If it comes, scope the product venture deliberately; if it doesn't, keep making EDM OS excellent for EDM.

One sentence to run on: build internal-first against real projects, hold the cheap disciplines (now implemented), defer the expensive product machinery, and let evidence — not a feeling of "finished" — decide whether EDM OS becomes a product.

EDM OS · Construction operating system for EDM Holdings · Dubai, UAE
Companion: the edm-os repository — configuration layer, feature flags and tenant scoping now implemented.
Build what you win, when you win it. Keep the option open cheaply. Sell on evidence, not on a feeling.